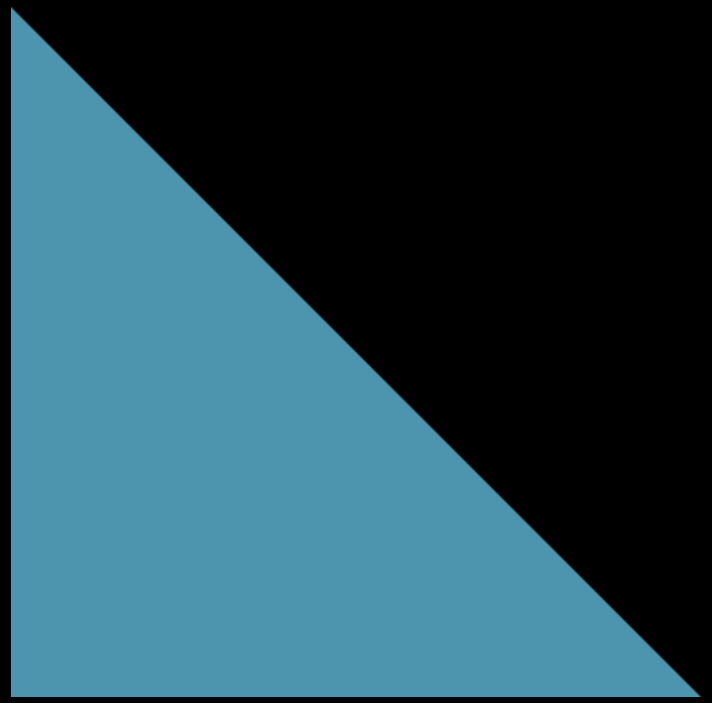
3D Graphic with OpenGL: Setting Up and Understanding the Environment



Provided to You By: Mario Aguilera Piceno

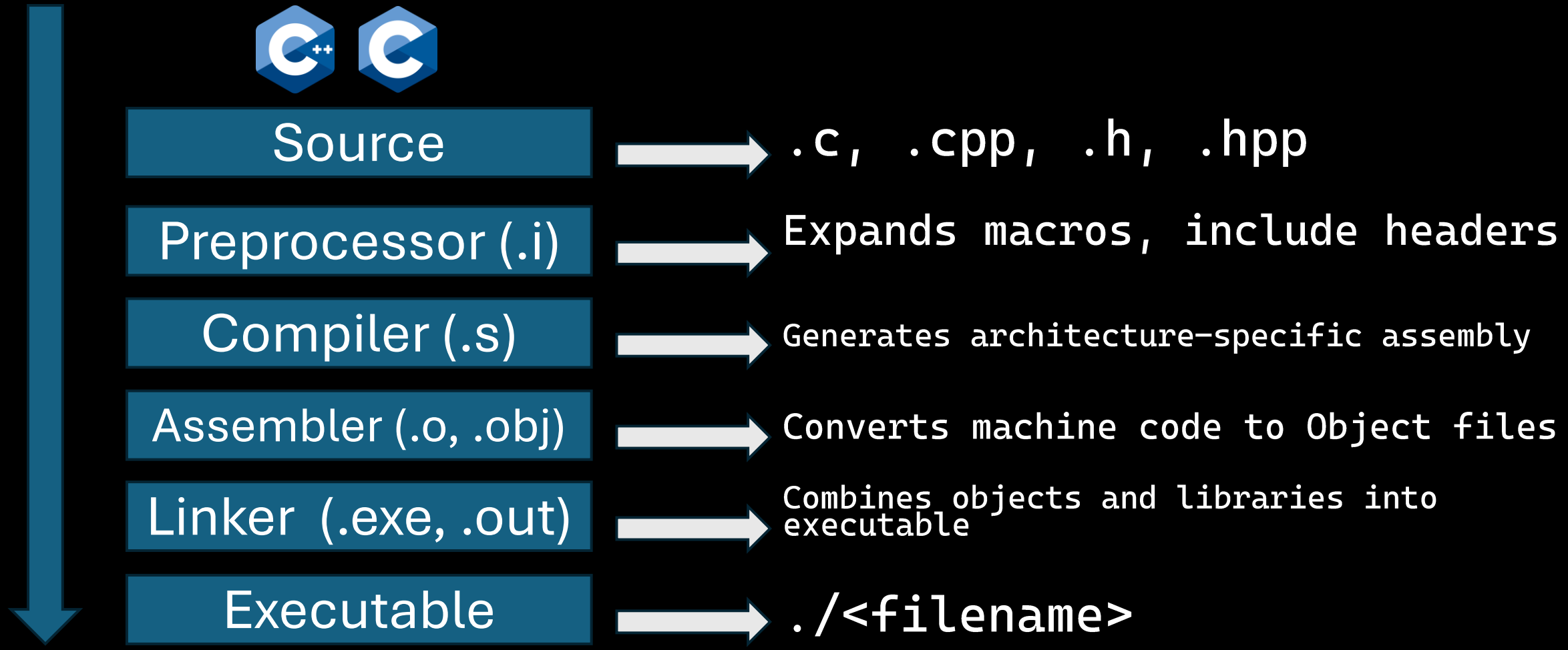
Understanding Compiling

- Compiling is the process of: compilation.
- It takes all source code file (C, C++, header files, etc.) and looks for code definitions and proper programming syntax (Does not check for runtime errors usually – compiler dependent).
- It creates an executable file using all provided source files

Understanding Compilation Process: C, C++

1. Source code: (C/C++ code written by programmer), .c, .h .cpp, .hpp files
2. Preprocessing: Preprocessor handles all lines starting with # (i.e. #include, #define), and replaces them with code from the included files, it expands on macros, and manages conditional compilation (#ifdef, etc.). This creates a .i file and passes to compiler
3. Compilation: Compiler translates C++ code into assembly and evaluates syntax and converts to assembly code (.s file)
4. Assembly: Assembler converts assembly code into object code, creating binary file (.o file)
5. Linking: Linker compiles any provided object files and links libraries (.h, .hpp) to create an executable (looks for main() method)
6. Executing: final object executable file can be called using ./<file>

Understanding Compilation Process: C, C++ - Diagram (Pipeline)



Typical Compilation Errors

1. Pre-processing: Header not found, macro errors
2. Compilation: Syntax errors, type mismatches
3. Assembly: Architecture errors
4. Linking: Undefined references, missing symbols
5. Execution: Segmentation faults, logic bugs

Understanding Dynamic-Linked Libraries (DLLs)

- Contains compiled code loaded and executed at runtime
- File extension (Windows): .dll

Understanding Static Libraries

- These are used at compile/link time
- Can contain compiled code (static library) or be a placeholder map to a DLL (import library) - Placeholder because compiler needs to understand that methods are referenced (i.e. prior to linking and utilizing a DLL)
- Also known as: Import Library(s) because they can serve as a placeholder for accessing methods after compilation. (During compilation, existence of all methods used need to be acknowledged, so methods do need to be implemented just yet)

Understanding Static Libraries. . .

- Problems: Whole app needs to be recompiled to update a static library.
- File extensions (Static): .lib (Windows), .a (Linux)
- File extensions (Import): .lib (Windows), .so (Linux)

Understanding Compilers

- MinGW (A C language) compiler that supports C++ and that is used for compiling C and C++ projects
- MSVC (Microsoft Visual C++) is a C and C++ compiler. It is the golden-standard for compiling almost any software application hosted on the Microsoft Windows OS.

Getting Started

- Software we will use:
  - Visual Studio
- Libraries we Will Use:
  - GLEW (OpenGL Extension Wrangler Library): https://github.com/nigels-com/glew
  - GLFW (OpenGL FrameWork): https://github.com/glfw/glfw
  - GLM (OpenGL Mathematics): https://github.com/g-truc/glm
  - SOIL2 (Simple OpenGL Image Library): https://github.com/SpartanJ/soil2

Understanding the Essentials (C, C++)

- C:
  - Created in 1972-1973 by Dennis Ritchie, who was working at AT&T Bell Laboratories
  - A programming language made to provide a high-level (less writing, more understanding) programming language that can provide low-level access to hardware. Alternative to assembly (second-gen language, made to replace binary coding)
  - This language forms the basis for many modern computer operating systems today.
- C++:
  - Created by Bjarne Stroustrup in 1979, while working at AT&T Bell Laboratories
  - “Often referred to as C with classes”
  - Classes are ways of creating “objects” to conduct “object-oriented” programming practices while providing low-level access and operations like C.
  - C++ has remained mostly backwards compatible with C, and you can import C libraries directly into C++ code using #include.
  - C++’s has since it’s creation, allowed for efficient modular programming, and it has become essential to game engine and 3D graphics programming

GLEW: Who, What, Where, When, Why



- Who: Created by Milan Ikits and Marcelo Magallon, maintained currently by Nigel Stewart
- What: Cross-platform C and C++ extension loading library. Available and tested on multiple major OS distributions (i.e. Windows and Linux). Reduces boilerplate code for accessing and utilizing GPU specific functions. Checks if a system supports certain OpenGL versions.
- Where: Milan created while also working on his PhD thesis at Scientific Computing and Imaging Institute
- When: Originated in 2002
- Why: As a way to provide run-time functionality for managing supported OpenGL extensions on a target system/platform.

GLFW: Who, What, Where, When, Why



- Who: Created by Marcus Geelnard. Maintained today by the GLFW Development Team
- What: Open-source library for OpenGL and Vulkan software. It handles window management, input/output, events, and contexts and surfaces of applications. Supports major OS platforms (i.e. Windows, Linux). Programmed in C99 (C language).
- Where: Sweden
- When: Created in 2002
- Why: As a way to provide simple window management for graphics applications.

GLM: Who, What, Where, When, Why



- Who: Created by Christophe Riccio (French programmer)
- What: C++ header math library for computer graphics (based on OpenGL Shading Language specs). Allows for linear algebraic operations for 3D graphics manipulation. Requires C++ 17 or newer.
- Where: France
- When: Created in 2005
- Why: As a way to provide a mathematic library for 3D graphics sizing, rotating, positioning.

SOIL2: Who, What, Where, When, Why



- Who: Created by Martín Lucas Golini based on Jonathan Dummer's SOIL project.
- What: Used for uploading textures into OpenGL. (Texturing models). Programmed in C language.
- Where: Argentina
- When: Created in 2017
- Why: As a way to provide a 3D model texturing method (PNGs, JPGs, etc.)

Overview Context: GLEW, GLFW, GLM, SOIL2

GLEW: Handles C and C++ extensions loading for run-time OpenGL extension support determining and loads them for use

GLFW: Handles Window management and interaction

GLM: Handles the mathematics for 3D graphics

SOIL2: Handles the image asset loading onto 3D graphics via OpenGL

Overview Diagram: GLEW, GLFW, GLM, SOIL2

GLEW 

GLFW 

GLM 

SOIL2 

Graphics Pipeline: Purpose

- Purpose: Provides a filter-based step-by-step process that is dependent (reliant) on the previous step to work as expected
- It takes all 3D processing instructions and runs it through the pipeline for rendering 3D graphics
- Why: It is essential for managing a 3D graphics software application

Graphics Pipeline: Simplified Diagram Overview